PartGuard - System Architecture Suggestions

for the following system, suggest suitable architectural styles/patterns to use.

PartGuard is an on-device mobile application that analyzes automotive repair receipts and verifies whether billed part prices exceed the manufacturer's suggested retail prices (MSRP). The system allows users to capture or upload repair receipts, automatically extract part numbers and associated prices, and compare them against official pricing data published by manufacturers such as General Motors.

Show more

You're describing a system that is actually very clean architecturally—if you structure it right, it'll stay maintainable as you scale from GM → multi-manufacturer → smarter extraction.

I'll give you **practical, non-theoretical guidance**: what you should actually use and why.

---

1. Core Architectural Style (Foundation)

✅ Layered Architecture (DI-style or Clean Architecture)

This should be your backbone.

Why?

- You already have **distinct responsibilities**:
  - UI (capture receipt)
  - OCR + parsing
  - price verification logic
  - data access (MSRP dataset)
- You want **strict separation** because:
  - OCR will evolve
  - pricing sources will change
  - logic must remain stable

Suggested Layers

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

- **Presentation Layer**
  - Camera / upload UI
  - Highlight results (overpriced parts)
- **Application Layer (Use Cases)**
  - `AnalyzeReceipt`
  - `VerifyPartPrices`
- **Domain Layer**
  - `Part`, `ReceiptItem`, `PriceComparison`
  - Core logic: *overcharge detection*
- **AI Processing Layer**
  - OCR (Vision framework)
  - (future) ML extraction model
- **Data Layer**
  - MSRP datasets (GM pricing)
  - External API (optional later)

👉 This is basically your **DIAMM-style architecture**, and it fits perfectly.

---

2. Key Architectural Patterns (What actually matters)

🔷 (1) Pipeline Pattern (CRITICAL)

Your system is inherently a pipeline.

```
Image → OCR → Text → Extract Parts → Match MSRP → Compare → Result
```

Why this matters:

- Keeps processing modular
- Easy to debug each stage
- You can swap components independently

Implementation hint:

Each stage = **stateless processor**

```swift
protocol ProcessingStep {
    func process(input: T) -> U
}
```

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

🔷 (2) Strategy Pattern (for OCR & Ex

You will NOT stick with one extraction method.

Use it for:

- OCR engines (Apple Vision vs future model)
- Extraction logic (rule-based vs ML)

```
ExtractionStrategy
  ├── RuleBasedExtractor
  ├── MLBasedExtractor (future)
```

👉 This prevents your system from collapsing when rules break.

🔷 (3) Repository Pattern (for MSRP Data)

You'll have multiple pricing sources:

- Local dataset (offline)
- Cached data
- API (future)

Use:

```
PartPriceRepository
  ├── LocalCSVRepository
  ├── CachedRepository
  ├── RemoteAPIRepository
```

👉 Keeps your domain logic completely clean.

🔷 (4) Adapter Pattern (VERY IMPORTANT)

Manufacturer data formats will differ.

GM today → others tomorrow.

Use adapters:

```
GMPriceAdapter
FordPriceAdapter
ToyotaPriceAdapter
```

👉 Normalize into:

```swift
struct PartPrice {
    partNumber: String
    msrp: Double
}
```

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

🔷 (5) Template Method (for Receipt Processing)

All receipts follow the same **high-level flow**, but details differ.

```
ProcessReceipt:
  1. OCR
  2. Normalize text
  3. Extract items
  4. Validate
```

👉 Subclasses customize extraction rules.

🔷 (6) Specification Pattern (for Validation Rules)

Your "overcharge detection" will evolve.

Today:

- price > MSRP

Tomorrow:

- % threshold
- regional adjustments
- labor vs part filtering

Use:

```
OverpricedSpecification
ThresholdSpecification
ValidPartSpecification
```

👉 Clean rule composition without hardcoding logic.

🔷 (7) Facade Pattern (for simplicity)

Expose ONE clean API to UI:

```Swift
PartGuardAnalyzer.analyze(image)
```

Internally:

- pipeline
- repositories
- strategies

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

👉 Keeps UI dead simple.

---

3. Data & Matching Patterns (Important for your idea)

🔷 (8) Dataset Matching Pattern

You are doing:

> extracted part number → match against dataset

This is NOT trivial in real life:

- OCR errors
- formatting differences

You need:

- normalization (strip spaces, dashes)
- fuzzy matching (later)

👉 Start simple:

- exact match (MVP)
- then evolve to embedding / similarity (like your MenuCal)

🔷 (9) Caching Pattern

MSRP data won't change frequently.

Use:

- local cache
- periodic refresh

👉 Avoid hitting network every time.

---

4. On-Device AI Considerations

You are essentially building:

✅ AI-assisted deterministic system

- OCR = AI
- Extraction = rule-based (for now)
- Matching = deterministic

👉 This is GOOD. Don't overcomplicate early.

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

5. What NOT to do (important)

Don't:

- ❌ Mix OCR logic with business logic
- ❌ Hardcode GM-specific parsing everywhere
- ❌ Tie UI directly to extraction logic
- ❌ Assume receipts are clean (they're messy)

---

6. Recommended Minimal Architecture Stack

If I had to force you into a clean MVP:

MUST HAVE:

- Layered Architecture
- Pipeline Pattern
- Repository Pattern
- Strategy Pattern

ADD NEXT:

- Adapter Pattern
- Specification Pattern

LATER:

- ML extraction
- fuzzy matching
- multi-manufacturer support

---

7. Reality Check (straight advice)

Your idea is solid—but the **hard part is NOT OCR**.

It's:

- messy receipts
- inconsistent part formats
- unreliable extraction

If you don't architect for flexibility now, you'll re

---

8. Bottom Line

If you do this right, PartGuard becomes:

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature

> A modular, pipeline-driven, on-device AI-assisted verification system with pluggable extraction strategies and dataset-backed validation.

That's a **strong, scalable architecture**—not just a quick app.

If you want, next step I can:

- map this into **actual Swift module structure**, or
- align it explicitly with your **DIAMM paper style** so you can reuse it academically.

CHATGPT TO PDF V1.4.1

Theme and code export fixes

- Improved code text clarity when export and site themes differ.
- Fixed long inline code export so content expands instead of being clipped in a scroll area.
- Fixed table export when exporting to Word.

Have an idea? Request a feature